UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

INFORME DE LABORATORIO

PRUEBAS AUTOMATIZADAS, PERSISTENCIA Y CI

Nombre(s): Carlos Hernan Garza Torres Oscar Denilson García Vélez Edwin Benjamín Ramírez Nava	
Matrícula(s): 1986937 1994273 1956899	
Unidad 5 · Calidad y verificación de software	
Nombre del profesor: Miguel Ángel Gómez Marroquin	
Proyecto analizado: Proyecto_Python_Base_U5	Tecnologías: Python, pytest, pytest-cov, SQLAlchemy, SQLite y GitHub Actions

Fecha: 02/11/2026

unidos por un cambio transformador

Índice

- 1. Introducción
- 2. Objetivos
- 3. Material proporcionado y alcance
- 4. Tipos de pruebas: unitarias, integración y E2E
- 5. Arrange–Act–Assert y ciclo TDD
- 6. Análisis funcional del proyecto
- 7. Matriz de casos CP01–CP15
- 8. Persistencia SQLAlchemy: commit y rollback
- 9. Simulación de cobros mediante mocks
- 10. Autorización y reglas de cancelación
- 11. Ejecución y resultados de pruebas
- 12. Cobertura de código
- 13. Pipeline de integración continua (CI)
- 14. Evidencia reproducible
- 15. Observaciones y mejoras recomendadas

La evidencia de ejecución también muestra un ResourceWarning por una conexión SQLite que no se cerró explícitamente. Aunque las pruebas terminan aprobadas, conviene cerrar sesiones y motores mediante context managers o fixtures para evitar ese aviso.

- 16. Conclusión

1. Introducción

Las pruebas de software permiten comprobar que una aplicación cumple sus reglas de negocio, conserva correctamente la información y responde de manera controlada ante errores. En esta práctica se estudia un proyecto didáctico de gestión de pedidos desarrollado en Python. El proyecto incorpora lógica de dominio, servicios de aplicación, persistencia con SQLAlchemy, validaciones de autorización y pruebas automatizadas con pytest.

El análisis se realizó a partir del archivo LEEME_INSTRUCCIONES.txt y de los archivos contenidos en CLN_U5_Archivos_Completos_Laboratorio.zip. Se revisaron los módulos fuente, los archivos de pruebas, los CSV de datos ficticios, el archivo SQLite de demostración y el workflow de GitHub Actions. Además, se ejecutó la suite incluida para obtener resultados observables de aprobación y cobertura.

Es importante distinguir la evidencia ejecutada localmente de la ejecución remota en GitHub Actions: el paquete contiene la definición del pipeline, pero no incluye un registro de una ejecución remota ni un commit identificable. Por ello, este informe describe cómo comprobar el pipeline y qué evidencia debe conservarse, sin afirmar que ya se ejecutó en un repositorio remoto.

2. Objetivos

- Distinguir pruebas unitarias, de integración y de extremo a extremo (E2E).
- Aplicar el patrón Arrange–Act–Assert para estructurar pruebas legibles.
- Comprender el ciclo TDD Red–Green–Refactor mediante el caso CP08.
- Verificar la persistencia y la reversión de transacciones SQLAlchemy sin utilizar datos productivos.
- Simular cobros mediante mocks, comprobar autorización y revisar cobertura.
- Interpretar el pipeline CI y documentar un procedimiento reproducible.

3. Material proporcionado y alcance

Elemento	Contenido y función
LEEME_INSTRUCCIONES.txt	Define instalación, ejecución, uso de CSV/SQLite, particularidad de CP08 y entregables.
Proyecto_Python_Base_U5.zip	Proyecto Python con módulos src/, pruebas tests/, requirements.txt y workflow CI.
src/domain.py	Entidad Order, descuento VIP, validación de total y deuda, confirmación.
src/application.py	OrderService; coordina confirmación y cobro por medio de un gateway.
src/repository.py	Modelo SQLAlchemy OrderRow, repositorio y creación de motor SQLite.
src/security.py	Función can_cancel para decidir si un usuario puede cancelar un pedido.
tests/	Pruebas unitarias y de integración implementadas con pytest.
pedidos_prueba.csv / clientes_prueba.csv / casos_de_prueba.csv	Datos didácticos para escenarios CP01–CP15; no se importan automáticamente en pytest.
pedidos_prueba.sqlite3 y cargar_datos_demo.py	Base SQLite de demostración y script que la reconstruye desde CSV.

El README aclara que el ejemplo es un proyecto de pedidos y no constituye la solución de las 17 empresas del PIA. También especifica que el pipeline compila el código como verificación de build, pero no realiza un despliegue productivo.

El repositorio utiliza SQLite en memoria para sus pruebas de repositorio. Los CSV y la base pedidos_prueba.sqlite3 son material didáctico opcional y no se cargan automáticamente. Esta

separación evita que una prueba modifique accidentalmente una base de datos externa o productiva.

4. Tipos de pruebas: unitarias, integración y E2E

Tipo	Qué verifica	Ejemplo en este proyecto	Alcance
Unitaria	Una función o unidad aislada, normalmente con dependencias controladas.	test_vip_discount, test_customer_debt y pruebas de can_cancel.	Rápida; comprueba reglas del dominio o autorización.
Integración	La colaboración entre componentes o con una infraestructura real de prueba.	test_repository_commit y test_repository_rollback con SQLAlchemy y SQLite en memoria.	Comprueba mapeo ORM, sesión y transacciones.
Extremo a extremo (E2E)	Un flujo completo desde una interfaz o punto de entrada hasta sus efectos finales.	No hay prueba E2E implementada en el paquete. Un E2E futuro podría crear un pedido, autorizarlo, cobrarlo y consultar su persistencia mediante una API o interfaz.	Comprueba el sistema como lo usaría un usuario; requiere más infraestructura.

No debe llamarse E2E a una prueba de servicio que invoca directamente una clase. En el proyecto actual, test_service.py comprueba OrderService con un gateway simulado; es una prueba unitaria del servicio con mock, no un flujo E2E. Las pruebas de repositorio sí son de integración porque utilizan SQLAlchemy y una base SQLite real, aunque efímera.

5. Arrange–Act–Assert y ciclo TDD

5.1 Patrón Arrange–Act–Assert (AAA)

AAA organiza cada prueba en tres partes: Arrange (preparar datos, objetos y dependencias), Act (ejecutar la operación que se quiere comprobar) y Assert (verificar el resultado). Separar estas etapas facilita entender qué condición se está probando y por qué falla.

```
def test_vip_discount():
    # Arrange
    order = Order(100, vip=True)

    # Act
    amount = order.confirm()

    # Assert
    assert amount == 85
    assert order.status == "CONFIRMED"
```

En el ejemplo se prepara un pedido VIP de 100; se confirma y se comprueba que el descuento del 15 % deja un importe de 85 y que el estado cambia a CONFIRMED.

5.2 TDD: Red–Green–Refactor

Etapa	Actividad	Aplicación al caso CP08
RED	Escribir primero una prueba que exprese el comportamiento deseado y comprobar que falla.	Crear un pedido, confirmarlo una vez y volver a confirmar. La segunda confirmación debe rechazarse. Con el código base actual, la prueba nueva falla porque Order.confirm() no impide confirmar un pedido ya confirmado.
GREEN	Modificar lo mínimo necesario para que la prueba pase.	Añadir una validación del estado inicial NEW antes de confirmar; si ya está CONFIRMED, lanzar una excepción definida para esa condición.
REFACTOR	Mejorar diseño y legibilidad sin cambiar el comportamiento comprobado.	Extraer validaciones o mensajes comunes, mantener pruebas verdes y volver a ejecutar toda la suite.

El archivo LEEME_INSTRUCCIONES.txt advierte expresamente que CP08 está incluido para aplicar TDD: el código actual permite confirmar dos veces. Por tanto, no debe presentarse como un defecto de la ejecución de pytest inicial; es un comportamiento pendiente de cambiar mediante la prueba primero.

```
# Ejemplo orientativo de prueba RED para CP08.
def test_order_cannot_be_confirmed_twice():
    order = Order(250)
    order.confirm()

    with pytest.raises(ValueError):
        order.confirm()
```

Con la implementación entregada, esta prueba adicional debería fallar en RED porque el método no verifica el estado actual. Después de implementar la regla y repetir la prueba, debe pasar en GREEN. El código de ejemplo usa ValueError como alternativa sencilla; el equipo puede definir una excepción de dominio específica si documenta esa decisión.

6. Análisis funcional del proyecto

Módulo	Responsabilidad	Reglas identificadas
domain.py / Order	Representa un pedido y aplica reglas de confirmación.	El total debe ser mayor que cero; un cliente con deuda no puede confirmar; el cliente VIP recibe 15 % de descuento; al confirmar cambia a CONFIRMED.
application.py / OrderService	Coordina confirmación y cobro.	Llama order.confirm(); solicita gateway.charge(importe); si el gateway rechaza el pago, lanza RuntimeError("payment rejected").
repository.py	Persiste pedidos mediante SQLAlchemy.	OrderRow almacena id, total y status; SqlOrderRepository.save agrega la fila a la sesión; commit y rollback los controla la transacción de sesión.
security.py / can_cancel	Evalúa si se permite cancelar.	No permite cancelar pedidos SHIPPED; permite al administrador cancelar otros estados; para no administradores exige ser propietario y estar dentro de 24 horas.

La regla de cancelación tiene una prioridad importante: un pedido SHIPPED no puede cancelarse ni siquiera por un administrador, porque esa condición se evalúa antes de la excepción administrativa. Las pruebas de seguridad comprueban expresamente este comportamiento.

7. Matriz de casos CP01–CP15

La matriz siguiente relaciona los escenarios incluidos en casos_de_prueba.csv con la lógica y las pruebas disponibles. “Validado por suite” significa que existe una prueba automatizada equivalente en el paquete; no significa que el CSV se haya leído automáticamente. Los casos CP01–CP12 no están parametrizados desde el CSV en la suite entregada.

Caso	Entrada	Resultado esperado	Evidencia / prueba relacionada	Evaluación
CP01	Pedido regular 120	Confirmación y cobro 120.00	Equivalente cubierto por test_regular_no_discount	Cubierto por regla; CSV no cargado automáticamente
CP02	VIP 200	Confirmación y cobro 170.00	Equivalente cubierto por test_vip_discount	Cubierto por regla; CSV no cargado automáticamente
CP03	Pedido con deuda 500	CustomerHasDebt	test_customer_debt	Cubierto
CP04	Total cero	ValueError	test_invalid_total[0]	Cubierto
CP05	Total negativo -10	ValueError	test_invalid_total[-1]	Cubierto
CP06	VIP 1000	Confirmación y cobro 850.00	Equivalente de descuento VIP	Cubierto por regla; caso de importe no parametrizado
CP07	VIP 0.01	Confirmación y cobro 0.01 según CSV	No hay prueba específica de límite decimal	Pendiente de prueba específica
CP08	Pedido ya	No permitir segunda	No existe en suite inicial; el código	Pendiente intencional; iniciar RED

	CONFIRMED	confirmación (objetivo TDD)	permite confirmar dos veces	
CP09	VIP 300	Confirmación y cobro 255.00	Equivalente de descuento VIP	Cubierto por regla; CSV no cargado
CP10	Regular 75	Confirmación y cobro 75.00	Equivalente de pedido regular	Cubierto por regla; CSV no cargado
CP11	VIP con deuda 250	CustomerHasDebt	test_customer_debt	Cubierto por regla; combinación VIP/deuda no parametrizada
CP12	VIP 99.99	Confirmación y cobro 84.99	No hay prueba específica de este importe	Pendiente de prueba específica
CP13	Gateway rechaza el pago	RuntimeError: payment rejected	test_gateway_rejected	Cubierto
CP14	Persistencia con commit	Registro visible después de commit	test_repository_commit	Cubierto
CP15	Persistencia con rollback	Registro ausente después de rollback	test_repository_rollback	Cubierto

Para cubrir CP01–CP12 de manera sistemática se recomienda agregar pruebas parametrizadas que lean pedidos_prueba.csv con csv.DictReader y comparen importe, estado o excepción. La guía indica que los datos no se cargan solos. Debe documentarse la conversión de los campos numéricos y booleanos del CSV, así como el resultado calculado.

8. Persistencia SQLAlchemy: commit y rollback

El repositorio usa SQLAlchemy ORM. make_engine() crea por defecto un motor sqlite+pysqlite:///:memory: y crea las tablas declaradas en Base. Cada prueba crea su propio motor y una sesión. Así, la prueba opera sobre una base temporal en memoria, no sobre datos de producción.

8.1 Commit

En test_repository_commit se guarda un pedido con id 1 y total 120, se ejecuta session.commit() y luego se consulta OrderRow. El resultado esperado es encontrar el registro con total 120. Esto demuestra que la transacción fue confirmada y que la sesión puede leer el dato persistido.

```
engine = make_engine()  # SQLite en memoria
with Session(engine) as session:
    SqlOrderRepository(session).save(1, Order(120))
    session.commit()
    row = session.scalar(select(OrderRow).where(OrderRow.id == 1))
    assert row.total == 120
```

8.2 Rollback

En test_repository_rollback se agrega el mismo tipo de fila, pero se ejecuta session.rollback() en lugar de commit. La consulta posterior debe devolver None. Esto demuestra que los cambios pendientes se revierten y que el registro no queda guardado.

```
engine = make_engine()  # SQLite en memoria
with Session(engine) as session:
    SqlOrderRepository(session).save(1, Order(120))
    session.rollback()
    row = session.scalar(select(OrderRow).where(OrderRow.id == 1))
    assert row is None
```

La seguridad de esta estrategia depende de mantener la URL de prueba en memoria y no sustituirla por credenciales o una URL de producción. No se debe ejecutar el script de carga de datos contra una base real. El archivo pedidos_prueba.sqlite3 y cargar_datos_demo.py son recursos didácticos separados; el script elimina y vuelve a crear tablas, por lo que debe ejecutarse únicamente sobre la base de demostración.

9. Simulación de cobros mediante mocks

Un mock sustituye una dependencia externa para controlar su comportamiento durante la prueba. En este caso, el gateway de pagos se simula con unittest.mock.Mock. No se realiza ningún cargo bancario ni se necesita conexión con un proveedor de pagos.

```
from unittest.mock import Mock

def test_gateway_approved():
    gateway = Mock()
    gateway.charge.return_value = True
    result = OrderService(gateway).checkout(Order(100))
    assert result == 100
    gateway.charge.assert_called_once_with(100)

def test_gateway_rejected():
    gateway = Mock()
    gateway.charge.return_value = False
    with pytest.raises(RuntimeError):
        OrderService(gateway).checkout(Order(100))
```

El primer escenario verifica que el servicio devuelve el importe confirmado y que invoca charge exactamente una vez con ese importe. El segundo simula un rechazo y comprueba que el servicio comunica el fallo mediante RuntimeError. El mock aísla la lógica de aplicación y hace que la prueba sea rápida, determinista y segura.

10. Autorización y reglas de cancelación

La función can_cancel recibe el estado, propietario, usuario que solicita la acción, indicador de administrador, fecha de creación y fecha actual opcional. Sus pruebas cubren cuatro situaciones:

- Propietario dentro de las 24 horas: se permite cancelar.
- Usuario distinto del propietario y no administrador: se deniega.
- Pedido SHIPPED incluso con administrador: se deniega.
- Pedido con más de 24 horas para un usuario no administrador: se deniega.

La fecha actual se inyecta en las pruebas para evitar depender del reloj del sistema. Esto hace reproducible la condición temporal. El caso de administrador sobre un pedido no enviado se permite por la implementación; sería recomendable agregar una prueba específica para fijar también ese comportamiento.

11. Ejecución y resultados de pruebas

Se ejecutó la suite incluida en Proyecto_Python_Base_U5 desde Windows, utilizando el entorno virtual del proyecto. Las capturas aportadas muestran Python 3.15.1, pytest 9.2.0 y plataforma win32. Esta ejecución local es distinta de la versión Python 3.11 fijada en el workflow de CI; por ello, la evidencia local no sustituye una ejecución del pipeline con la versión configurada.

```
python -m pytest -v --cov=src --cov-report=term-missing --cov-fail-under=80
```

Métrica observada	Resultado
Pruebas recopiladas	13
Pruebas aprobadas	13
Pruebas fallidas	0

Cobertura total	98.00 %
Umbral requerido	80 %
Resultado del umbral	Cumplido
Módulo con menor cobertura	src/security.py: 88 %
Entorno observado	Windows (win32)
Python / pytest	Python 3.15.1 / pytest 9.2.0
Instrucciones medidas	50; 1 no cubierta

Salida resumida observada:

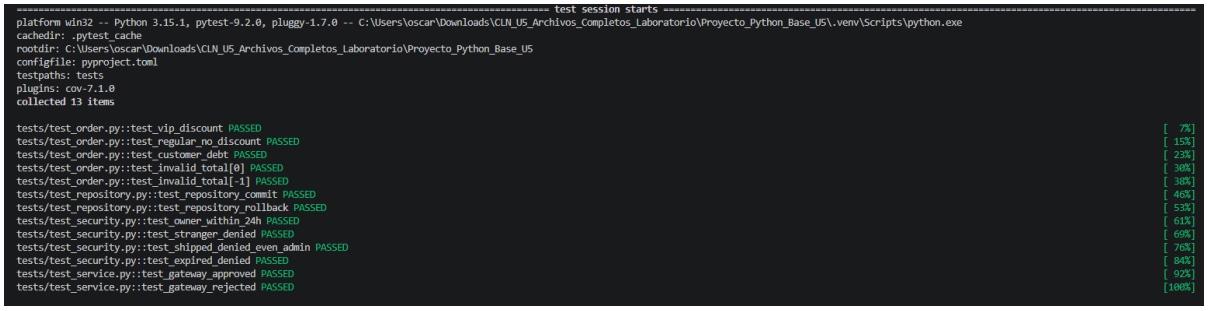


Figura 1. Ejecución de pytest: se recopilan 13 pruebas y las 13 aparecen aprobadas.

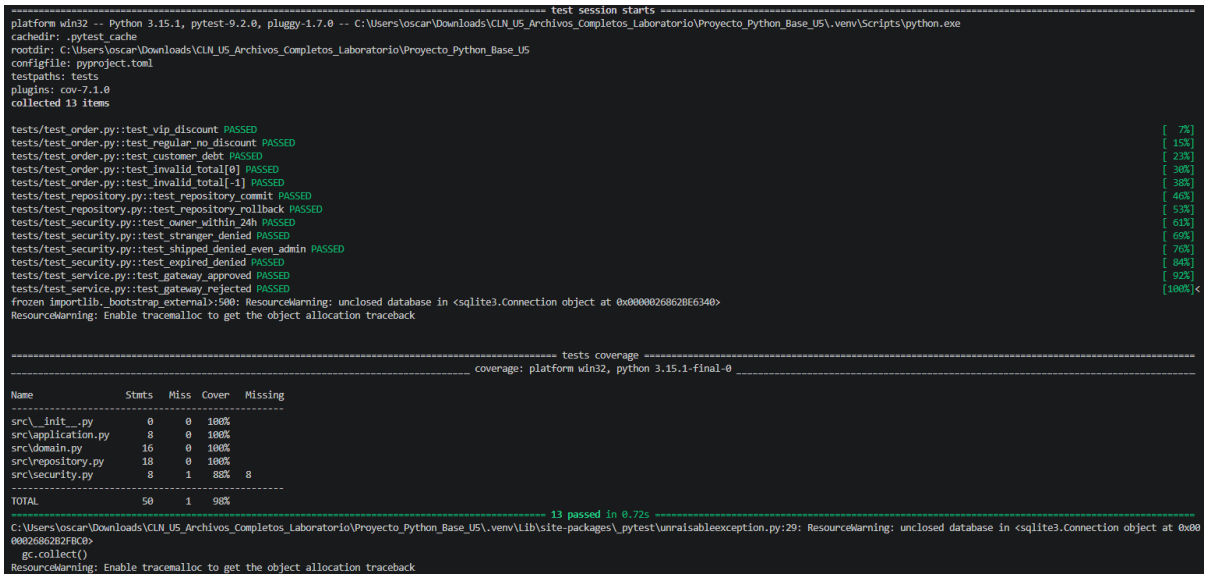


Figura 2. Ejecución con cobertura: 13 pruebas aprobadas y 98 % de cobertura. La salida también muestra un ResourceWarning de una conexión SQLite sin cerrar.

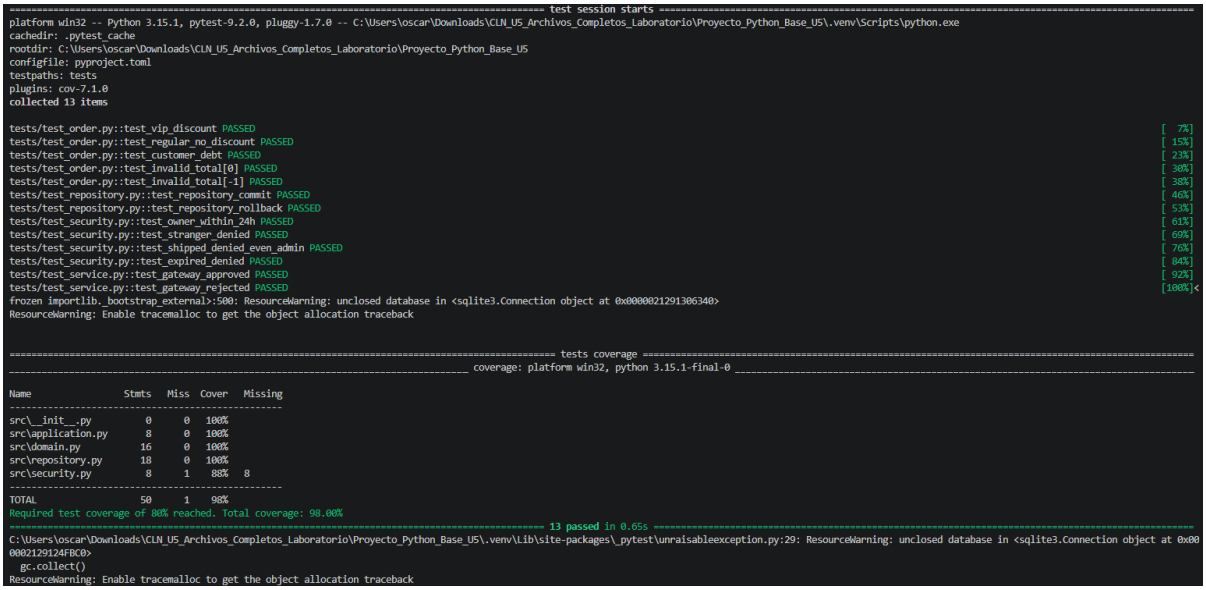


Figura 3. Resultado de la ejecución en Windows: 13 pruebas aprobadas, 98.00 % de cobertura y umbral mínimo del 80 % cumplido.

```
13 passed in 0.65s

Name                Stmts   Miss  Cover
src/__init__.py         0      0   100%
src/application.py      8      0   100%
src/domain.py          16      0   100%
src/repository.py      18      0   100%
src/security.py         8      1    88%
TOTAL                  50      1    98%

Required test coverage of 80% reached.
Total coverage: 98.00%
```

La línea no cubierta de security.py corresponde a la rama que genera automáticamente la fecha actual. En las pruebas se inyecta la fecha para controlar el tiempo. La cobertura observada en las capturas es 98.00 %; aunque supera el umbral solicitado, no demuestra por sí sola que todos los requisitos estén probados. CP08 y algunos límites de CSV requieren pruebas adicionales.

12. Cobertura de código

La cobertura es una medida de cuánto código se ejecutó durante las pruebas. pytest-cov integra pytest con coverage.py y permite generar un resumen por módulo. El parámetro --cov-fail-under=80 establece un mínimo: si la cobertura total queda por debajo del 80 %, el comando termina con error.

- --cov=src: mide el código dentro del paquete src.
- --cov-report=term-missing: imprime el porcentaje y las líneas no ejecutadas.
- --cov-fail-under=80: convierte el mínimo del 80 % en una condición de aprobación.

No se debe confundir cobertura con calidad absoluta. Una línea puede ejecutarse sin comprobar todos sus resultados posibles. Es conveniente complementar la métrica con casos límite, escenarios de error, pruebas parametrizadas y pruebas de integración.

13. Pipeline de integración continua (CI)

El archivo .github/workflows/ci.yml define un workflow llamado CI Business Logic. Se activa en push y pull_request, por lo que GitHub Actions debe ejecutarlo al enviar cambios o abrir/actualizar una solicitud de integración.

Paso del workflow	Qué hace	Qué significa aprobarlo
actions/checkout@v4	Descarga el código del repositorio.	El runner obtiene el contenido que se va a verificar.
actions/setup-python@v5, Python 3.11	Configura la versión de Python indicada.	La ejecución utiliza una versión controlada.
pip install -r requirements.txt	Instala pytest, pytest-cov, SQLAlchemy y ruff.	Las dependencias requeridas quedan disponibles.
ruff check src tests	Analiza estilo y errores estáticos detectables por Ruff.	No se encontraron problemas que incumplan las reglas configuradas.
pytest -v --cov=src --cov-report=term-missing --cov-fail-under=80	Ejecuta pruebas y mide cobertura.	Todas las pruebas pasan y la cobertura es al menos 80 %.
python -m compileall -q src	Compila sintácticamente los módulos fuente.	No se detectan errores de sintaxis en src.

El pipeline no despliega la aplicación ni ejecuta migraciones productivas. Tampoco demuestra por sí solo el comportamiento de una interfaz o API E2E. Es una verificación automatizada de calidad y compilación. Para considerar el CI realmente evidenciado en GitHub, debe existir una ejecución asociada a un commit o pull request y debe conservarse una captura o enlace al resumen verde de Actions.

14. Evidencia reproducible

14.1 Preparación en Windows

Descomprimir Proyecto_Python_Base_U5.zip en una carpeta independiente. Abrir PowerShell en esa carpeta y ejecutar:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
py -m pip install -r requirements.txt
```

Si PowerShell bloquea la activación del entorno, puede ejecutarse el intérprete del entorno directamente, sin activar:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pytest -v --cov=src --cov-report=term-missing --cov-fail-under=80
.\.venv\Scripts\ruff.exe check src tests
.\.venv\Scripts\python.exe -m compileall -q src
```

En Windows, si el comando python no está reconocido pero el lanzador py sí funciona, se puede utilizar py para crear el entorno e instalar dependencias. Una vez creado el entorno, conviene usar siempre .venv\Scripts\python.exe para garantizar que pytest utiliza las dependencias instaladas allí.

14.2 Evidencia que debe guardarse

- Capturas incluidas en este informe: recopilación de las 13 pruebas y ejecución local con cobertura de 98.00 %.
- Captura de la ejecución de ruff check src tests sin errores.
- Captura de compileall completado sin mensajes de error.

- Captura de las pruebas de commit y rollback aprobadas.
- Captura del workflow CI en GitHub Actions con el commit, fecha y estado de los pasos.
- Si se implementa CP08, captura de la prueba fallando antes del cambio (RED), pasando después (GREEN) y de la suite completa después del refactor.

Las capturas aportadas no muestran una ejecución verde final de Ruff, la salida de compileall, el ciclo TDD RED/GREEN/REFACTOR ni GitHub Actions remoto. Esas evidencias deben obtenerse al ejecutar dichos pasos; las capturas del editor del Anexo C muestran diagnósticos y sugerencias de corrección, no la confirmación de que Ruff terminó sin errores.

15. Observaciones y mejoras recomendadas

- Agregar pruebas parametrizadas con csv.DictReader para ejecutar automáticamente CP01–CP12 y comparar el resultado real con el esperado.
- Completar CP08 mediante TDD. La suite inicial no comprueba la doble confirmación y el comportamiento actual permite repetirla.
- Agregar pruebas específicas para CP07 y CP12, así como para la combinación VIP con deuda de CP11.
- Considerar Decimal para importes monetarios. El modelo actual utiliza float; en sistemas financieros reales es preferible Decimal para evitar errores de representación binaria.
- Agregar pruebas para la rama de can_cancel que genera la fecha actual cuando now no se proporciona y para administrador sobre un pedido no enviado.
- Cerrar explícitamente motores y sesiones en pruebas más amplias, usando context managers y fixtures de pytest.
- Agregar al menos una prueba E2E si el sistema incorpora una API o interfaz: enviar una solicitud real de prueba, comprobar respuesta, cobro simulado y persistencia.
- Proteger la base de demostración: ejecutar cargar_datos_demo.py solo sobre el archivo ficticio, ya que el script elimina y recrea tablas.

16. Conclusión

La práctica permitió identificar la función de las pruebas unitarias, de integración y E2E, así como la importancia de estructurarlas mediante Arrange–Act–Assert. El proyecto demuestra pruebas unitarias de reglas de pedidos, autorización y servicio de cobro, además de pruebas de integración para persistencia SQLAlchemy. Los mocks permiten simular la aprobación o rechazo de pagos sin conectarse a un servicio financiero real.

La ejecución local documentada con las capturas del equipo obtuvo 13 pruebas aprobadas de 13 y una cobertura total de 98.00 %, superior al mínimo solicitado del 80 %. También se describen las pruebas de commit y rollback dentro de SQLite en memoria, sin tocar datos productivos. El análisis detectó que CP08 es una actividad TDD pendiente, ya que el código base permite confirmar dos veces. Finalmente, el workflow CI automatiza instalación, análisis estático, pruebas, cobertura y compilación. Para cerrar la evidencia de entrega siguen pendientes la ejecución remota del pipeline y las capturas RED/GREEN/REFACTOR de CP08.

Anexo A. Archivos relevantes

Ruta	Uso
src/domain.py	Reglas del pedido y confirmación.
src/application.py	Servicio de checkout y gateway.
src/repository.py	Persistencia ORM SQLAlchemy y motor SQLite.
src/security.py	Reglas de autorización para cancelación.
tests/test_order.py	Pruebas de dominio.
tests/test_service.py	Pruebas del servicio y mocks.
tests/test_repository.py	Pruebas de commit y rollback.
tests/test_security.py	Pruebas de autorización.
.github/workflows/ci.yml	Definición del pipeline CI.
pedidos_prueba.csv, clientes_prueba.csv, casos_de_prueba.csv	Datos ficticios para ampliar las pruebas.
cargar_datos_demo.py	Regenera la base SQLite didáctica desde los CSV.

Anexo B. Comandos principales

Anexo C. Evidencia complementaria del código y las pruebas

Las siguientes capturas muestran archivos fuente y pruebas abiertas en el editor, junto con diagnósticos y sugerencias de Ruff para ordenar imports y actualizar el uso de UTC. Se incluyen como evidencia de revisión del código; no deben interpretarse como una captura de Ruff finalizado sin errores.

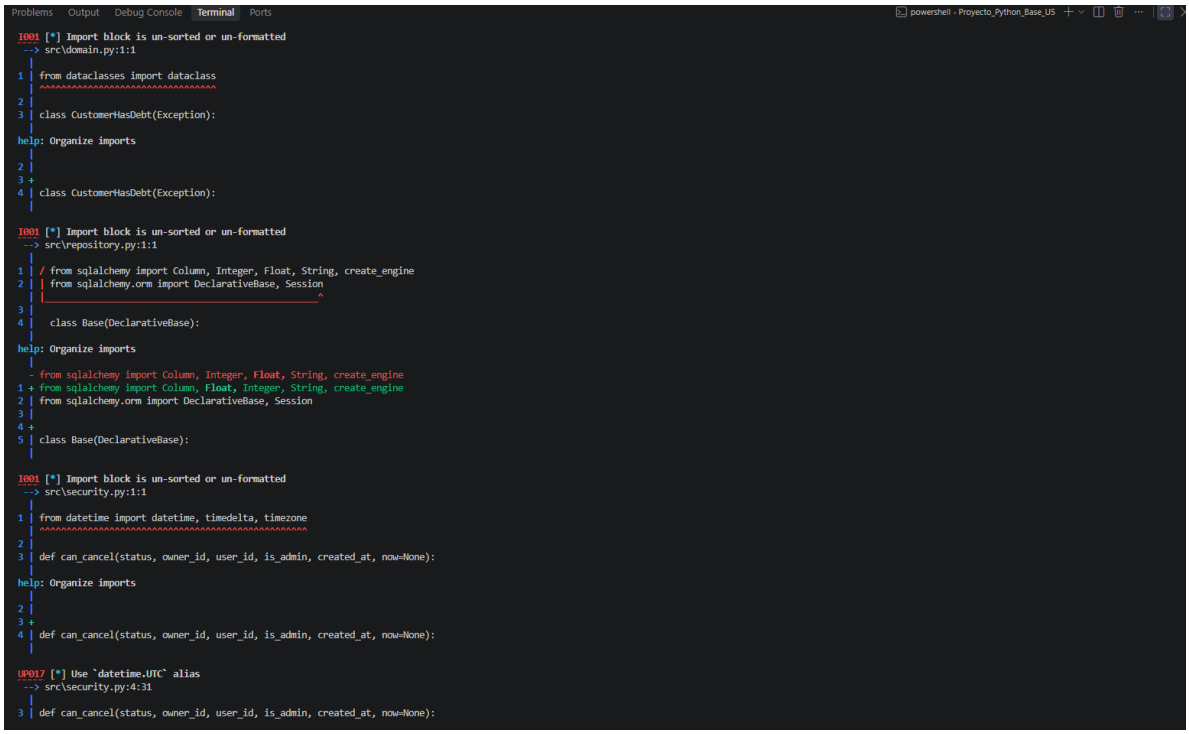


Figura 4. Revisión de módulos de dominio, repositorio y seguridad: Ruff señala imports desordenados o sin formato y propone organizarlos.

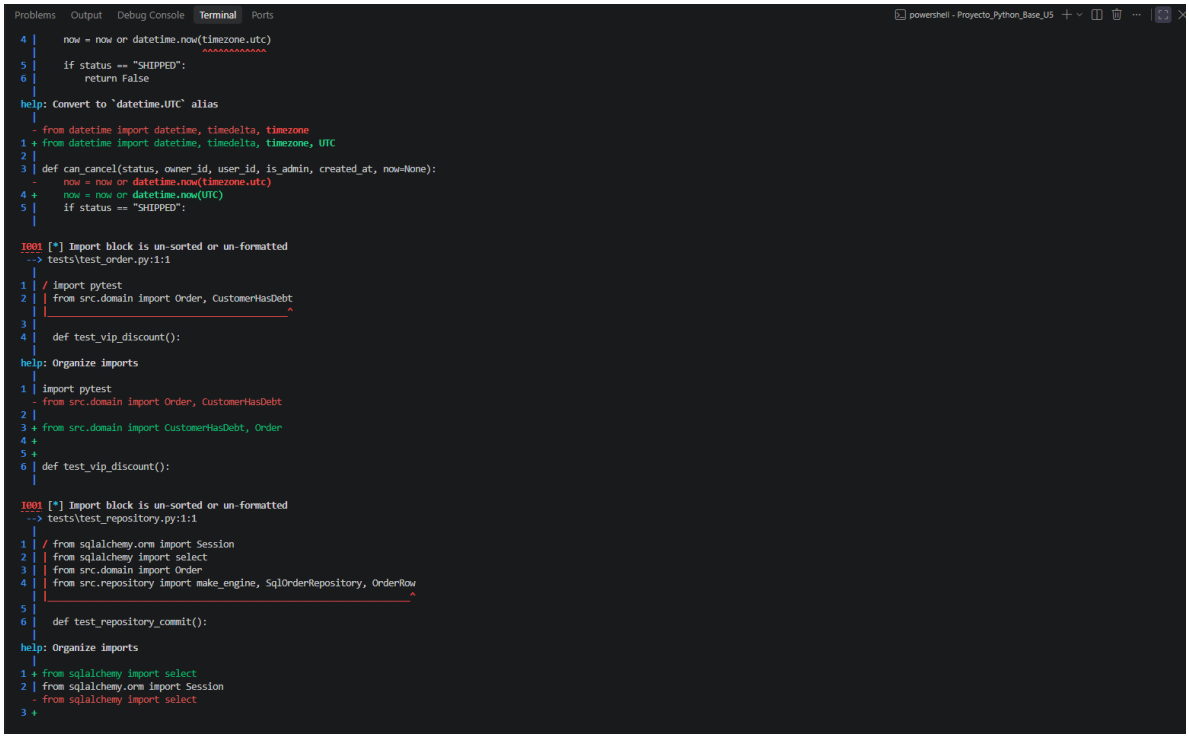


Figura 5. Revisión de security.py y de las pruebas de dominio y repositorio; se observan sugerencias de organización de imports y uso de datetime.UTC.

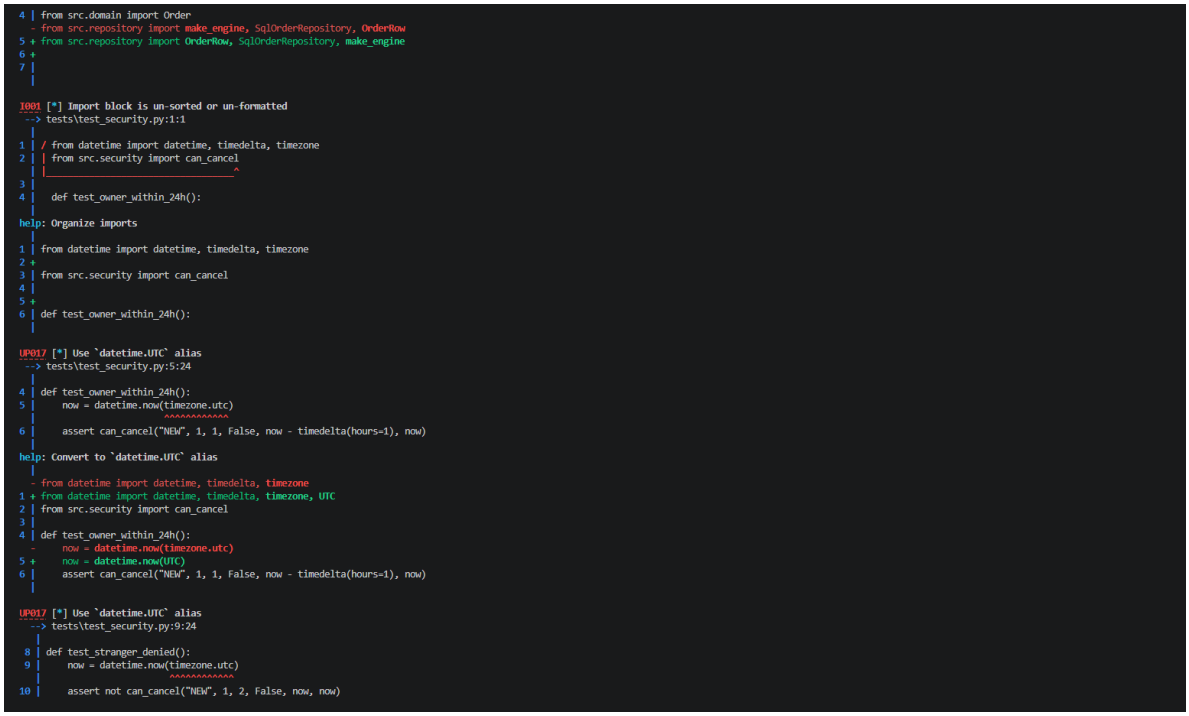


Figura 6. Revisión de las pruebas de repositorio y de autorización; se muestran casos de propietario y validaciones temporales.

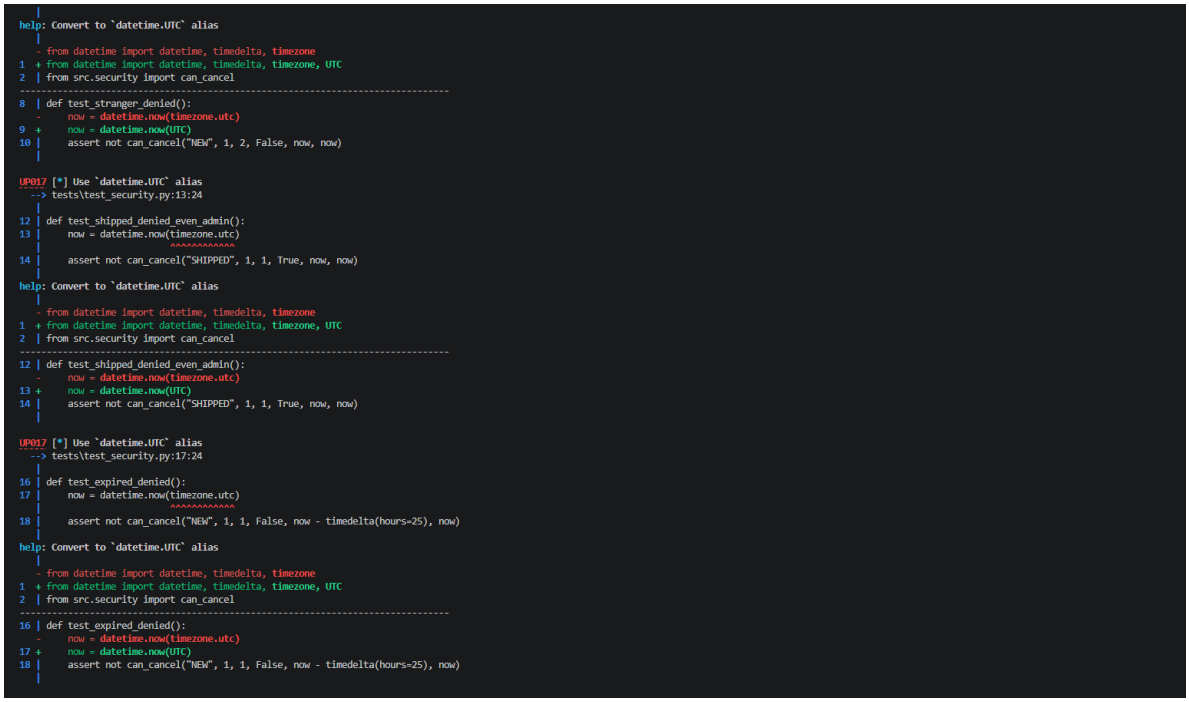


Figura 7. Continuación de las pruebas de autorización: usuario distinto, pedido enviado y pedido fuera del límite de 24 horas.

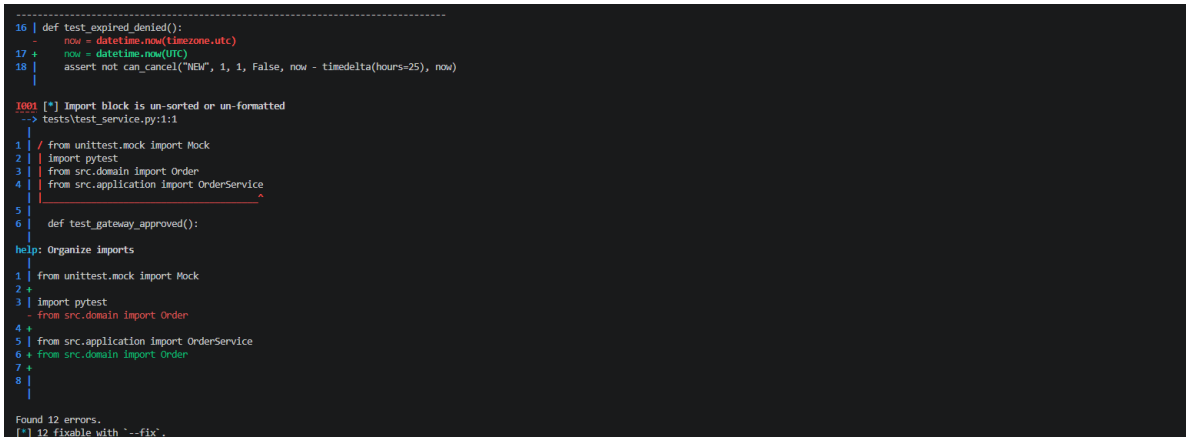


Figura 8. Revisión de test_service.py: prueba del gateway de pagos simulado y organización de sus imports.

```
# Instalar dependencias
py -m pip install -r requirements.txt

# Ejecutar pruebas y cobertura
py -m pytest -v --cov=src --cov-report=term-missing --cov-fail-under=80

# Análisis estático
ruff check src tests

# Verificación de compilación
py -m compileall -q src

# Regenerar SOLO la base didáctica local (opcional)
py cargar_datos_demo.py
```